

DT124A - Lab 3 report

Name: Gagan Adhikari

Date: 2025/11/06

Authorship statement:

Gagan Adhikari declares that this lab report is a result of *my* own original work. Any outside sources used have been properly cited and acknowledged. This report has not been submitted previously for any academic or professional purpose.

AI has been used only to refine text on some manner only.

Table of Contents

1. Introduction 4

2. Materials and Methods 4

3.1 Task 1 4

3.1.1 Discussion 4

3.2.1 Flowchart

3.2.2 pseudo code

3.2.3 Test Cases

3.2 Task 2

3.2.1 Discussion , flowchart, pseudo code , Test cases

References 15

1. Introduction

There are 2 tasks in the Lab 3,

Task 1:

Diving competitions are based on a point system in which divers are scored on factors including execution and degree of difficulty.

Task 2:

Computation of e^x

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \ldots = \sum_{n=0}^{\infty} \frac{x^n}{n!},$$

Pow() and fabs() should not be used.

2. Materials and Methods

Only Computer are being used not, any hardware is used.

3. Result

3.1 Task 1

3.1.1 Discussion

These are the basic Top-Down Design.

Goal: Compute diver's score from 5 judges and degree of difficulty.
Inputs:

- Five scores (floating-point values) from judges.
- Degree of difficulty (floating-point).

Outputs:

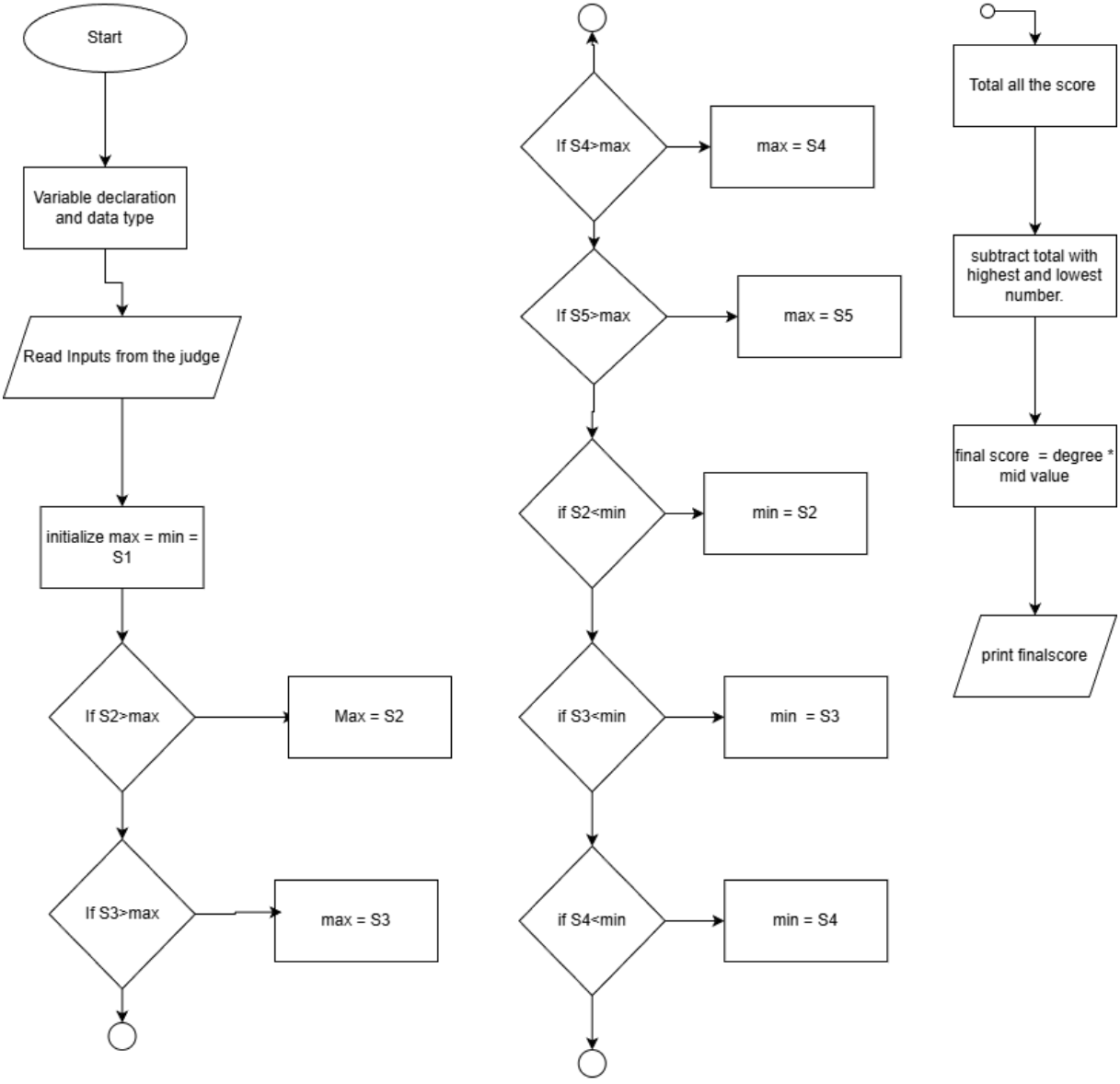
- Final score (floating-point).

Major Steps (Top-Down):

1. Read 5 scores and degree of difficulty.
2. Identify the highest and lowest scores.

3. Discard them.
4. Sum the remaining 3 scores.
5. Multiply the sum by degree of difficulty.
6. Output the result.

3.2.1 Flowchart



3.2.2 Pseudo-Code

```
BEGIN
   READ score1, score2, score3, score4, score5
   READ degree_of_difficulty
 SET highest = max(score1, score2, score3, score4, score5)
 SET lowest = min(score1, score2, score3, score4, score5)
 SET sum = (score1 + score2 + score3 + score4 + score5) - highest - lowest
 SET final_score = sum * degree_of_difficulty
PRINT final_score
END
```

3.2.3 Test Case

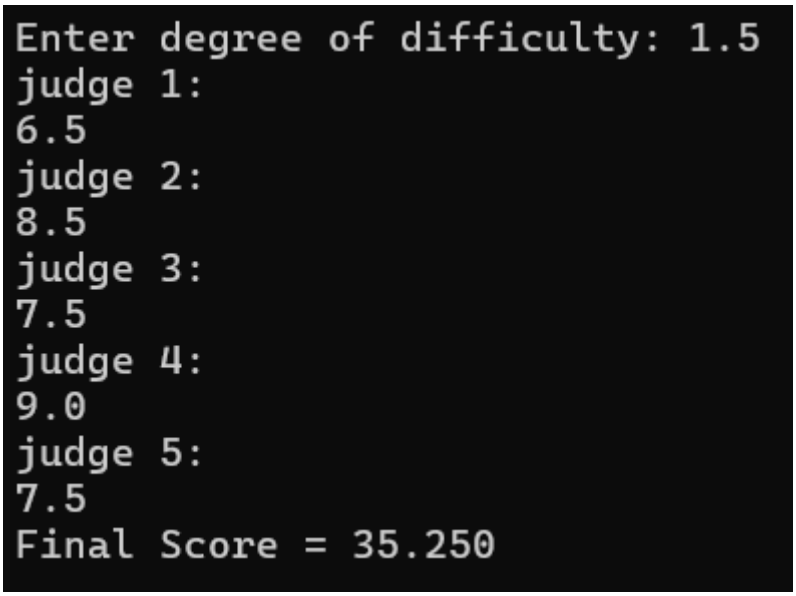
Here are some Test Cases and their result:

Test 1:

5 judge score are: 6.5, 8.5, 7.5, 9.0, 7.5

And degree is: 1.5

Result:



Test 2:

5 judge score are: 5.0, 5.0, 5.0, 5.0, 5.0

And Degree is: 2.0

Result:

```
Enter degree of difficulty: 2
judge 1:
5
judge 2:
5
judge 3:
5
judge 4:
5
judge 5:
5
Final Score = 30.000
```

Here is Code Implementation:

```c
int main()
{
    float s1,s2,s3,s4,s5;//inputs variables
    float de_diff,min,max,total,mid,finalscore;//variables

    printf("Enter degree of difficulty: ");
    scanf("%f",&de_diff);
    printf("judge 1: \n");
    scanf("%f",&s1);
    printf("judge 2: \n");
    scanf("%f",&s2);
    printf("judge 3: \n");
    scanf("%f",&s3);
    printf("judge 4: \n");
    scanf("%f",&s4);
    printf("judge 5: \n");
    scanf("%f",&s5);
    //find highest number and minimum
    max = min = s1;   // start with s1 as both max and min

    if (s2 > max) max = s2;
    if (s3 > max) max = s3;
    if (s4 > max) max = s4;
    if (s5 > max) max = s5;

    if (s2 < min) min = s2;
    if (s3 < min) min = s3;
    if (s4 < min) min = s4;
    if (s5 < min) min = s5;

    //total including all number
    total = s1+s2+s3+s4+s5;
    // middle numbers
    mid = total - max - min;

    finalscore = mid * de_diff;

    printf("Final Score = %.3f\n", finalscore);
    return 0;
}
```

3.2 Task 2

Compute e^x

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \ldots = \sum_{n=0}^{\infty} \frac{x^n}{n!},$$

where n! denotes the factorial of n. This Taylor series is an infinite series and it is not possible for you to implement an infinite loop to compute the function. In practice, it is enough to add the first n terms.

Top-Down Design

Goal: Approximate e^x using the Taylor series expansion.

Inputs:

- A number x (integer in your current program, but conceptually real).

Outputs:

- A table showing partial sums after 1, 2, 3, … terms.
- Final approximation of e^x.

Major Steps (Top-Down):

1. **Read Input**
    - Prompt user for x.
2. **Initialize Variables**
    - Set sum = 0, n = 0.
3. **Loop Until Convergence**
    - Compute current term: \frac{x^n}{n!}.
    - If absolute value of term < 10^{-7}, stop.
    - Add term to sum.
    - Print current approximation.
    - Increment n.
4. **Output Final Result**

- Print final approximation of e^x

Implementation of code

```c
#include <stdio.h>
#include <stdlib.h>

// factorial function
double facto(int x)
{
    double fact = 1.0;
    for (int i = 1; i <= x; i++)
    {
        fact = fact * i;
    }
    return fact;
}

// power function (x^n)
double power(int x, int n)
{
    double p = 1.0;
    for (int i = 1; i <= n; i++)
    {
        p = p * x;
    }
    return p;
}
```

```c
double main()
{
    int x, n = 0;
    printf("Enter the value of x: ");
    scanf("%d", &x);

    double term = 1.0;
    double sum = 0.0;

    printf("Terms\tApproximation\n");

    while (1)
    {
        // compute current term: x^n / n!
        term = power(x, n) / facto(n);

        // stop if term is very small (absolute value)
        if (term < 0.0000001 && term > -0.0000001)
        {
            break;
        }

        sum = sum + term;

        printf("%d\t%.7f\n", n + 1, sum);

        n++;
    }

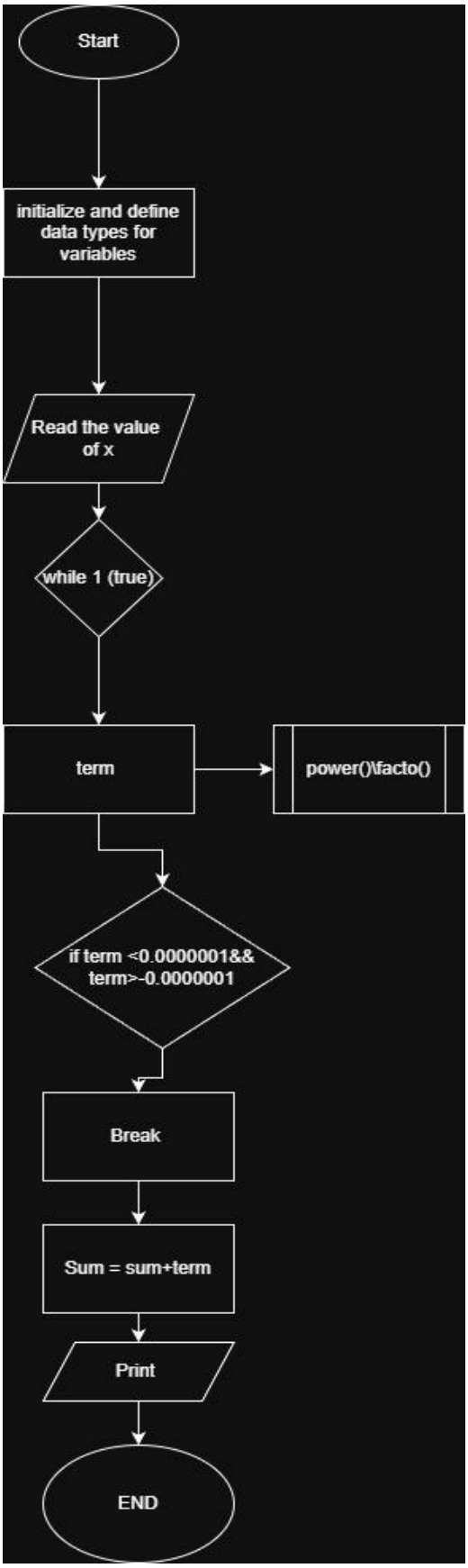
    return 0;
}
```

This code is having comment on each line

This C program computes an approximate value of e^x using its (taylor, n.d.). It repeatedly calculates each term of the series using the functions power() for x^n and facto() for n!. Starting with n = 0, it evaluates each term as x^n/n!, adds it to a running sum, and prints the current approximation after every iteration. The loop continues until the term becomes extremely small (less than 0.0000001), at which point further terms would have negligible impact on the result, so the loop stops. This method allows the program to approximate e^x with progressively increasing accuracy until the desired precision is reached. (taylor, n.d.)

I would like to mention one thing after some tweak inn my previous code it can now calculate the e^-x also i.e. negative numbers. But there are some overflows in between even Tho it gives the correct output at last.

Flowchart:


Start
initialize and define data types for variables
Read the value of x
while 1 (true)
term
power()\facto()
if term <0.0000001&& term>-0.0000001
Break
Sum = sum+term
Print
END

Test Case

X = 2,

```
Enter the value of x: 2
Terms   Approximation
1       1.0000000
2       3.0000000
3       5.0000000
4       6.3333333
5       7.0000002
6       7.2666667
7       7.3555558
8       7.3809528
9       7.3873020
10      7.3887129
11      7.3889951
12      7.3890465
13      7.3890552
14      7.3890595
15      7.3890724
16      7.3890888
17      7.3891213
```

```
Enter the value of x: 4
Terms   Approximation
1       1.0000000
2       5.0000000
3       13.0000000
4       23.6666667
5       34.3333327
6       42.8666654
7       48.5555538
8       51.8063471
9       53.4317437
10      54.1541422
11      54.4430999
12      54.5481761
13      54.5832012
14      54.6179368
15      54.8278233
16      55.3635401
```

X=-1

```
Enter the value of x: -1
Terms   Approximation
1       1.0000000
2       0.0000000
3       0.5000000
4       0.3333333
5       0.3750000
6       0.3666667
7       0.3680556
8       0.3678571
9       0.3678819
10      0.3678792
11      0.3678795
```

References

factorial. (n.d.). *wikipedia*. Retrieved from wikipedia: https://en.wikipedia.org/wiki/Factorial

taylor. (n.d.). *wikipedia*. Retrieved from https://en.wikipedia.org/wiki/Taylor_series

wikipedia. (n.d.). https://en.wikipedia.org/wiki/Factorial.